

CSE-CIC-IDS2018 — สรุปการเทรนทุกรอบ

9 บันทึกลับ เรียงตามลำดับที่หา ทุกบันทึกใช้ 15 คลาส และ 77 feature (ตัด Dst Port, Timestamp และคอลัมน์ระบุตัวตนออก)

ผลดิบอยู่ที่ `results/ids2018/<bundle>/` โมเดลอยู่ที่ `models/ids2018/<bundle>/`

ตารางรวม

บันทึก	ขนาด	วิธีแบ่ง	ทุน	ตัวนำ	macro-F1
300k	300k	สุ่ม 70/30	ไม่	stacking	0.6886
300k_balanced_weighting	300k	สุ่ม 70/30	ไม่	lightgbm	0.6953
300k_temporal	300k	ตามเวลา 70/30	ไม่	catboost	0.7277
300k_temporal_tuned	300k	ตามเวลา 70/30	ใช่ ไม่แฟร์	catboost	0.7751
300k_temporal_tuned_fair	300k	ตามเวลา 70/30	ใช่ แฟร์	catboost	0.7751
500k	500k	สุ่ม 70/30	ไม่	lightgbm	0.7311
500k_temporal	500k	ตามเวลา 70/30	ไม่	lightgbm	0.7318
500k_temporal_tuned	500k	ตามเวลา 70/30	ใช่ ไม่แฟร์	lightgbm	0.7321
500k_temporal_tuned_fair	500k	ตามเวลา 70/30	ใช่ แฟร์	random_forest	0.7678

บันทึกที่ควรใช้อ้างอิงคือสองตัวที่ทุนแบบแฟร์ ส่วนตัวอื่นเก็บไว้เพื่อดูว่าอะไรเปลี่ยนอะไร

สิ่งที่แต่ละรอบตอบ

300k และ 500k — ฐานเริ่มต้น

แบ่งข้อมูลแบบสุ่ม ไม่ทุน ใช้ดูว่าโมเดลทำได้แค่ไหนโดยไม่ปรับอะไร

300k_balanced_weighting — ทดลองแก้ class imbalance

ใช้ `class_weight` ถ่วงน้ำหนักคลาสหายาก ผลขยับจาก 0.6886 เป็น 0.6953 ซึ่งเล็กกว่าความกว้างของช่วงความเชื่อมั่นมาก รายละเอียดอยู่ใน `docs/imbalance_experiments.md`

*_temporal — เปลี่ยนไปแบ่งตามลำดับเวลา

แบ่งตามเวลาในทุกกลุ่ม (วันที่จับข้อมูล, คลาส) แทนการสุ่ม เพื่อไม่ให้ flow จากช่วงการโจมตีเดียวกันไปอยู่ทั้งฝั่ง train และ test

*_temporal_tuned — ทุน hyperparameter

ทุนด้วย RandomizedSearchCV แต่รอบนี้ไม่แฟร์ ทุกโมเดลได้ลอง 20 ชุดเท่ากัน ทั้งที่ขนาด search space ต่างกันตั้งแต่ 10 ถึง 5,184 ชุด ผลคือ logistic_regression ได้ค้นครบทุกความเป็นไปได้ ส่วน xgboost ได้ค้นเพียง 0.4%

*_temporal_tuned_fair — แก้ให้ทุนแฟร์ □

ปรับ search space ของทั้งหกโมเดลให้เท่ากันที่ 144 ชุด ทุกตัวจึงได้ค้น 13.9% เท่ากัน

ผลของการแก้สำคัญกว่าตัวการแก้เอง อันดับของโมเดลเปลี่ยนไป 10 จาก 14 ตำแหน่ง แต่คะแนนบนคลาสที่วัดผลได้จริงไม่มีโมเดลไหนขยับเกิน 0.007

สองกรณีที่ชี้ชัดที่สุด

- mlp บน 300k ขึ้นจากอันดับ 7 เป็นอันดับ 3 (+0.1131 บนกระดาน แต่ +0.0027 บนคลาสจริง)
- lightgbm บน 500k จับได้ hyperparameter ชุดเดิมเป๊ะ คะแนนตรงกันทุกหลัก แต่ตกจากอันดับ 1 เป็น 2

catboost เป็นตัวควบคุม เพราะ grid ของมันไม่ถูกแก้ ผลจึงเข้าได้ถึงทศนิยมตำแหน่งที่ 10

ข้อควรระวังที่ใช้กับทุกบันเดิลของ 2018

สืคลาสมี test flow น้อยเกินกว่าจะวัดผลได้

ที่ขนาด 500k มีสืคลาสที่มี test flow รวมกันเพียง 25 แกวจาก 150,011 แกว

คลาส	test flow
SQL Injection	1
Brute Force -XSS	2
Brute Force -Web	6
DDOS attack-LOIC-UDP	16

แต่ละคลาสถือน้ำหนัก 1/15 ของ macro-F1 เท่ากับ Benign ที่มี 124,610 แกว อันดับบนกระดานจึงถูกตัดสินด้วย 25 แกวนี้

ผลคือช่วงความเชื่อมั่นของ macro-F1 กว้างเฉลี่ย 0.128 ที่ 300k และการเพิ่มข้อมูลเป็น 500k ไม่ได้ทำให้แคบลงเลย เพราะความกว้างมาจากคลาสที่หายากที่สุด ไม่ได้มาจากขนาดข้อมูลรวม

ต้องรายงาน macro-F1 สองแบบเสมอ ทั้งแบบครบ 15 คลาส และแบบเฉพาะ 11 คลาสที่มีข้อมูลพอ เมื่อดูเฉพาะ 11 คลาส โมเดลไม่เชิงเส้นทั้งหกอยู่ในช่วง 0.83 ถึง 0.85 ห่างกันไม่ถึง 0.02 มีเพียง logistic_regression ที่แยกออกมาที่ 0.78

สามคลาสที่ทุกโมเดลติดเพดานเดียวกัน

คลาส	ช่วง F1 จาก 7 โมเดล
Infiltration	0.00 ถึง 0.13
DoS attacks-SlowHTTPTest	0.619 ถึง 0.638
FTP-BruteForce	0.755 ถึง 0.800

เมื่อ logistic regression กับ gradient boosting ที่ทุนแล้วได้ผลเท่ากัน เพดานนั้นอยู่ที่ข้อมูล ไม่ใช่ที่โมเดล

ไฟล์ในโฟลเดอร์นี้

ไฟล์	เนื้อหา
data/all_bundles_model_comparison.csv	ผลทุกโมเดลทุกบันเดิลรวมกัน 62 แกว
data/per_class_<bundle>.csv	F1 รายคลาสของแต่ละบันเดิล

เอกสารอื่นที่เกี่ยวข้อง

- docs/runbook.md — วิธีรันและบทเรียนจากบันทึก 300k ไป 500k
- docs/imbalance_experiments.md — การทดลองแก้ class imbalance
- docs/model_comparison_ids2018_bilingual*.md — รายงานเปรียบเทียบโมเดลฉบับสองภาษา